

MDPM 883 · DESIGNING THE DIGITAL PRODUCT

PRISM Furniture Request

Task Descriptions Supporting the Hierarchical Task Model

Team Dart

Milestone 2 · Requirements Documents

Companion to the task model on the team Miro board

September 2026

Purpose

A hierarchical task model has to be readable at a glance, so the label on each node is short. This document fills in what that brevity leaves out. It takes every task in both hierarchies of the PRISM Furniture Request task model and says what the task involves, why the user does it, and which side of the product boundary it falls on. The purpose recorded on the board is to support a more consistent and reliable Furniture Request process, helping CMACs submit and follow requests while helping Infrastructure staff manage request information across groups and handoffs. These tasks are what that purpose looks like in practice.

Each hierarchy is set out as one table. The numbering shows parentage rather than sequence: 5.6.1 is part of 5.6 and not the step after it, and a requester may well locate the work before settling on the request type. Level 1 tasks are the major subgoals, and level 2 and level 3 tasks are what those break down into. Read downwards, each level answers the question “in order to do this, what do I need or wish to do?”

The Task column uses the label from the task model board word for word. The only changes are the capitalisation of the first letter and four spellings: “worn out” at 2.1, “exists” at 2.5 and 3.4.2, and “communication” at 5.6.2. The full stop the board carries at the end of 5.4 is dropped, as no other label has one. Where an earlier draft of this document and the board disagreed about a label, its wording or where it sat in the hierarchy, the board won.

The Where performed column says which side of the product boundary a task falls on. *Outside* is work done away from the system, such as judging that a chair is broken. *Partly outside* is work done elsewhere but recorded in the request. *In product* is a task the product has to support directly. This follows the scope note on the board: procurement, warehouse activities, installation and delivery sit outside the product, which may track their status without performing or managing them. Tasks done outside are still in the model, because they show how people reach their goal as they move between the product and other tools, and because the product has to be able to record what they conclude.

The Requester Goal — Task Hierarchy and Descriptions

The tasks below correspond to section 4.2 of the Requirements Documents deliverable and to the requester hierarchy on the task model board, and they carry the same numbers there as here, so a task can be cited once and found in either document. They cover the work of a Client Ministry Accommodation Contact or other authorized requester, from being able to use the service at all through to confirming that the result was satisfactory. The goal breaks into six subgoals: getting set up on the system, understanding the furniture need, locating where the work needs to be done, preparing and submitting the request, following it, and completing and closing it.

No.	Task	Level	Where performed	Description
Goal	Get the Furniture Request fulfilled	Goal	Partly outside	The requester wants a furniture need in their office turned into furniture that is in place and working. Everything below is done in service of that. The hierarchy is a decomposition rather than a sequence, so a requester may well locate the work before settling on the request type, and the goal is finished only when the requester has seen the result and accepted it, not when Infrastructure marks the work done.
1	Get setup on the system	1	Partly outside	Everything a requester has to sort out before they can raise a request at all: knowing this is the right service, having credentials for it, and understanding the language the form uses. Someone who requests often does this once and never thinks about it again. Someone who requests once a year has to re-establish all three each time, which is why it is a subgoal in its own right — a request that never gets started is invisible to the service.
1.1	Identify PRISM request as the right portal	2	Outside	Working out that a furniture need goes through the Furniture Request service and not somewhere else, such as a maintenance request, a procurement process, a service desk ticket or an email to a facilities contact. Occasional requesters often start in the wrong place and lose days before someone redirects them. The product's part is to be easy to find and to say plainly, at its front door, what it covers and what it does not.
1.2	Obtain an account / log in	2	Partly outside	Getting credentials that work and reaching the point where a request can be started, either by signing in with an existing government account or by being granted access for the first time. Most of this happens outside the product, in an access process the requester does not control. It still belongs here, because a requester who cannot sign in has no way of telling that apart from the service being down.
1.3	Understand form fields and terminology	2	In product	Working out what the form is actually asking for: what a work order is, how a modification differs from a repair, what counts as a building code, what the service means by a planning request. Requesters are administrative staff without a facilities background, so the vocabulary belongs to Infrastructure rather than to them. The product has to explain its own terms where they are used instead of assuming them.
1.3.1	Read hints	3	In product	Checking the guidance attached to a field while filling it in, rather than reading documentation first. Requesters prefer this because it costs nothing when it is not needed and it appears exactly where the doubt is. It also shows in what gets submitted: the fields that are explained are the ones that come back complete.

No.	Task	Level	Where performed	Description
2	Understand the furniture need	1	Outside	Working out what is actually needed before anything is typed in. The CMAC is usually relaying a problem someone else reported, so most of this is a conversation and a look at the furniture, away from any screen. The product cannot do the thinking, but each conclusion ends up as a field on the request, so it has to record them without insisting they were reached in a particular order.
2.1	Recognise the furniture problem (broken / worn out / new)	2	Outside	Judging whether an item is broken, worn out, unsuitable for the person using it, or simply missing because someone new has arrived. It is a judgement made by looking at the furniture, and it is the first fork in the model: it decides whether the answer is a repair, a warranty claim, a replacement or a new purchase, and therefore what evidence the request will need. A photograph or a short video taken at this point is what later settles any argument about what was wrong.
2.2	Determine the request type	2	Partly outside	Choosing among the request types the service supports — Purchase New, Reconfiguration, Ergonomics, Moves, Recycle, Combination, Greater than 9 Workstations and STIP, as FR-01 and the data dictionary list them. Each one asks for different supporting information, carries different mandatory attachments and takes a different route once it reaches Infrastructure. The requester decides outside the product and records the decision in the request. Choosing wrongly sends the request to the wrong group, and that usually only comes to light when a coordinator opens it.
2.3	Determine if it belongs to a group or an individual	2	Outside	Deciding whether the request is for one person's workstation or for a shared area such as a meeting room, a reception or a floor being reconfigured. That changes the quantity, the approval path and often the urgency, and it changes who has to be consulted first. It is kept as its own task because requesters regularly submit as one person's problem what is really an area-wide one, and putting that right costs a cycle.
2.4	Decide recycled vs new	2	Partly outside	Establishing whether the need can be met from government furniture already in stock before anyone considers buying. It is a cost decision and a stated policy preference at the same time, and the requester needs it settled early because it changes what they have to supply: a recycled item needs dimensions and a location, a new one may need a quote and an approval. The product's job is to carry that preference on the request — recycled preferred, or required — and keep it in front of the coordinator who acts on it. Kevin settles this before anything else in Scenario 3, when he needs fourteen chairs for an office in Calgary.

No.	Task	Level	Where performed	Description
2.4.1	Check recycle-program availability	3	Partly outside	Finding out whether the recycle program has something suitable: the right type, the right quantity, in reasonable condition, available when it is needed. In practice this means asking someone and waiting for an answer by email. The product should let the preference be stated and the answer be written back to the request, so the reason a request went to purchase stays on the record instead of in someone's memory.
2.5	Check whether a request already exists for the item / location	2	In product	Looking for an open request covering the same need before raising another one. This is the requester's own defence against duplicates, and it does not work today, because a requester cannot see requests raised by other authorized requesters for the same area. That is how Owen ends up with two requests for one chair in Scenario 2. Supporting it means letting a requester search the requests for the buildings they look after, within what their role permits, rather than only the ones they submitted themselves.
2.6	Check warranty / retrieve original specs	2	In product	Finding the original request for an item already in place, to work out whether it is still under warranty and what exactly was supplied: manufacturer, model, finish, dimensions, and the purchase and installation dates. This is what Hugh needs in Scenario 6, two years after the desk was installed, and it is why history sits inside the requester's goal rather than being treated as finished business. Without it, a replacement part gets ordered by guesswork, or a valid warranty claim never gets made.
3	Locate where work needs to be done	1	Partly outside	Describing the place precisely enough that the furniture arrives in the right room. Vagueness here causes most of the delivery errors the scenarios describe, and each one is paid for twice: once in the wrong delivery and again in the move that fixes it. The model breaks this down further than it might seem to need, because the building, the address, the floor, the area and the plan each go wrong on their own.
3.1	Identify the building	2	In product	Picking the Government of Alberta facility from the buildings the requester is authorized for. Everything else about the location hangs off it: the address, the floor plans that exist, and the service group responsible. That is why it is chosen from a list rather than typed.
3.2	Confirm address	2	In product	Confirming the street address that comes with the building rather than typing it again. The requester is checking, not entering, which is quicker for them and the only way the address on the request can be trusted to match the facility record it came from.

No.	Task	Level	Where performed	Description
3.3	Identify the floor and area	2	In product	Narrowing the location to a floor and then to an area, room or workstation on it. This is the precision the delivery and installation teams actually work to, and it is where the incomplete request in Scenario 7 falls over: a floor with no area named cannot be given to an installer, however complete the rest of the request is.
3.4	Provide floor plan and mark the location	2	In product	Showing exactly where the furniture goes by marking it on a plan of the area. Words handle this badly — which wall, which side of the partition, which of four identical workstations — and a marked plan settles it. It matters most for moves and reconfigurations, where both where the furniture is coming from and where it is going have to be shown — Lina marks two plans in Scenario 5, one for each building.
3.4.1	Select stored floor plan	3	In product	Using a plan the facility already has. It is quicker than making one and more accurate, because it is the plan the installers work from. The board shows this and the sketch beside it as siblings: they are two ways of doing the same parent task, not two steps.
3.4.2	Provide a sketch when no plan exists	3	Partly outside	Drawing the area by hand or photographing it when no plan is held. Plans are not kept for every area and a request should not stall for want of one, so a sketch photographed on a phone and attached will do, especially alongside the floor and area fields.
4	Prepare and submit the request	1	In product	Putting everything gathered so far into a request and sending it to Infrastructure. This is where the requester's work first becomes visible to the service, and where completeness counts most: anything left out turns into a question later, and every question adds a round trip that the requester feels as delay and the coordinator sees as queue.
4.1	Complete the intake form	2	In product	Recording the need, the request type, the location, the quantities, the priority and the reason for the request. The form is the requester's account of the problem, and it has to be fillable in one sitting by someone who does this once a year and has had no training.
4.2	Attach supporting files	2	In product	Adding the photographs, plans, quotes, approvals and checklists that show what is needed. Attachments carry a lot of the meaning of a furniture request: a photograph answers in a glance what a written description leaves open, and it survives the translation between the requester's words and Infrastructure's.
4.2.1	Upload multiple files at once	3	In product	Adding several files in one go instead of one at a time. A typical request carries a marked plan, a few photographs and an approval, and a one-at-a-time upload is the point where a requester decides that three photographs will do instead of six.

No.	Task	Level	Where performed	Description
4.2.2	Add a photo from mobile at time of request	3	Partly outside	Taking the photograph where the problem is, when it is noticed, and attaching it without moving it to a desktop first. This is why the product has to work on a phone: the requester knows most about the problem while standing in front of it, and anything that defers the photo loses detail.
4.2.3	Record and attach a short fault video	3	Partly outside	Showing a fault a still photograph cannot: a sit-stand desk that stops partway through raising, a chair mechanism that gives way under load, a drawer that jams. A few seconds of video saves a whole exchange of clarifying questions, and it often decides whether the fault is treated as a warranty claim or a repair.
4.2.4	Label each file	3	In product	Naming attachments so a coordinator, or the requester two years later, can tell what each one shows without opening all of them. Unlabelled files are how attachments stop being evidence and turn into clutter, and labelling is the cheapest way to prevent it.
4.3	Save a draft	2	In product	Putting a part-finished request aside and coming back to it. Requesters get interrupted, and they often have to go and find something — a floor plan, a quantity, an approval — before they can carry on. A request that cannot be paused is one that gets abandoned and started again from scratch.
4.4	Submit the request	2	In product	Sending the finished request for review in one deliberate action, with no separate status change to remember. Submitting also hands ownership to Infrastructure, so both sides need to be clear that it has happened.
4.5	Receive submission confirmation	2	In product	Getting a clear acknowledgement that the request arrived, with a reference the requester can quote. This closes the loop that currently has infrequent requesters submitting a second request because they are not sure the first one landed, which is one of the two ways duplicates get in.
4.6	Check that the request is complete before submitting	2	In product	Going over the assembled request against what the chosen request type needs: the marked plan for a reconfiguration, the approval for a purchase, the product detail for a warranty claim. On the board the task sits at the end of the branch, which is where requesters usually discover it mattered. The product's job is to move that discovery earlier, by saying what is missing while the request can still be fixed without going through a coordinator.
5	Follow request	1	In product	Everything the requester does between submitting and completion. This subgoal is the heart of the problem statement: it runs on email, phone calls and memory rather than on any system, so a requester's knowledge of their own request is only as good as their inbox, and it disappears entirely while they are away.

No.	Task	Level	Where performed	Description
5.1	See current status	2	In product	Finding out what state the request is in — created, submitted, assigned, completed — without asking anyone. It is the smallest useful thing the product can give a requester, and the one most likely to remove a phone call.
5.2	See workflow stage / next steps	2	In product	Understanding what is happening now, who has the request and what happens next, rather than just reading a status label. This is what a requester needs in order to answer the manager who asked them, and a label rarely does it: “in progress” does not say whether the request is waiting on a quote or on a delivery date.
5.2.1	See third party vendor progress	3	In product	Seeing how far an outside supplier, mover or installer has got: ordered, shipped, scheduled, installed. Suppliers do not work in the system, so this reaches the request through the staff who deal with them, and the task is here to make sure it gets there at all.
5.3	Determine if progressing or stalled	2	Partly outside	Working out whether silence means things are moving normally or that something has gone wrong. This judgement sets off every follow-up a requester makes, and today it is made on instinct, from how long it has been and what happened last time. Showing the stage, when it last changed and what is expected next turns the guess into a decision, and it is what lets Priya leave most of her requests alone in Scenario 4.
5.4	Stay informed about changes to the request	2	In product	Being told when something changes instead of going to look. What the requester really wants is to stop watching: to know that if anything happens they will hear about it, so that no news means nothing has happened.
5.4.1	Get immediate notification	3	In product	Being told promptly when the change needs something from the requester: a question from a coordinator, a delivery date to arrange access for, an approval that is waiting. Notifications that fire for everything get switched off, so the ones that matter have to look different from the ones that do not.
5.5	Respond to requests for additional information	2	In product	Supplying what a coordinator has asked for — a marked plan, a quantity, a product detail, a confirmation — and supplying it on the request rather than by return email. This is the requester’s half of Scenario 7, and it is where a request either becomes workable or goes back into the queue.
5.6	Communicate about the request	2	In product	Having the conversation a request generates — questions, clarifications, corrections, agreeing what was actually meant — in a way that stays with the request instead of spreading across email, Teams and phone calls.

No.	Task	Level	Where performed	Description
5.6.1	Reach the right person	3	In product	Getting to whoever holds the request now rather than whoever answered last time. Ownership changes at every handoff between service groups, and the requester has no way of knowing it has changed, so they write to the last name they have and wait.
5.6.2	Keep communication tied to request	3	In product	Making sure what was asked, answered and decided is written on the request rather than kept in a private mailbox. That is what lets the request survive the requester's absence, and it lets the next coordinator see why a decision was made.
5.6.3	Chase coordinator if/when stalled	3	Partly outside	Escalating when nothing has moved: phoning, emailing, asking someone to find out where the request has got to. If the rest of the model works this should become rare, and how often it happens is a fair measure of whether the product has succeeded.
5.7	Verify and correct onsite work	2	Partly outside	Checking what was delivered against what was asked for — the right items, the right quantity, the right room, assembled and working — and doing something about any difference. The check happens physically, in the office. The product's part is to record what it found and to let a correction be raised without starting over. The board puts it under following the request rather than under closing it, because a requester does this while the request is still live.
5.7.1	Confirm work was done correctly	3	Partly outside	Recording that the furniture arrived in the right place, in the right quantity and in working order. It is the requester's word rather than the installer's, and it is what separates work that was done from work that was accepted.
5.7.2	Flag a mistake	3	In product	Reporting a scope or installation error — wrong floor, wrong item, missing pieces, damage — on the existing request rather than opening a new one, so that the correction keeps the request's history, its location detail and its conversation. This is Kevin's wrong delivery in Scenario 1; raising a second request instead is exactly what loses the original intent.
6	Complete and close the request	1	In product	Bringing the request to a definite end, so that the record shows not just that the work happened but that the requester accepted it. Without an explicit close, a request is only assumed to be finished, and closing it becomes an administrative act done by whoever tidies the queue.
6.1	Confirm completion	2	In product	Saying that the need has been met: the furniture is in place, in the quantity agreed, and the person who reported the problem is working again. It is the only event in the model that can honestly end the requester's goal, and it is what makes the closed request worth trusting years later, when a warranty claim or a replacement part depends on it.

No.	Task	Level	Where performed	Description
6.2	Report a remaining issue	2	In product	Recording anything still outstanding when the request closes: a missing keyboard tray, a damaged panel, one chair short. Closure should not be a way of burying an unresolved problem. The board makes this a sibling of confirming completion rather than a new request, because the outstanding item belongs with the work that produced it.

Tasks not on the board

Four tasks from an earlier draft of this document are not on the board, so they are not rows above. Being added to the system as a user is an administrative act carried out by someone else. Identifying an origin and a destination is, on the board, two uses of 3.1 to 3.4 rather than a task of its own. Resolving a duplicate request and approving a change of supply path sit on the service staff side, at 1.4 and 2 of the second table, with the requester taking part through 5.5 and 5.6. The history and reporting tasks from that draft — finding an old request, listing active requests, reporting by location, status or period — are also absent from the board. What the requester needed them for survives at 2.5 and 2.6, the two points where a requester reaches back into past requests in order to raise a new one. The functional requirements on the board still carry all four: assigning and maintaining authorized roles, capturing origin and destination for move requests, resolving duplicates while preserving the history of the request that continues, and retrieving, searching and reporting on past requests. They are noted here so that leaving them off the hierarchy stays a decision someone made rather than something that slipped, and so that the model and the requirements are read together.

The Service Staff Goal — Task Hierarchy and Descriptions

The tasks below correspond to section 4.3 and cover the work of the Infrastructure staff who receive, coordinate and fulfil Furniture Requests. They complete the picture, because a request only ends well if both sets of tasks succeed. The table is numbered and laid out like the requester table above, so a task on one side can be read against the task it answers on the other: 5.5 against 1.3, 5.1 against 4.1, 6.1 against 5.1.

No.	Task	Level	Where performed	Description
Goal	Get the Furniture Request delivered for the client	Goal	Partly outside	The service staff goal: turning a submitted request into furniture in place, coordinated across whichever Infrastructure groups and outside suppliers the work needs, without the request losing coherence as it passes between them.
1	Take on an incoming request	1	In product	Everything between a request arriving and it becoming work someone can act on. The cost of an incomplete request lands here, which is why so much of the functional specification is about completeness at submission.
1.1	Review what has been submitted	2	In product	Reading the request, its attachments and its location detail to understand what is being asked for.

No.	Task	Level	Where performed	Description
1.2	Judge whether the request is complete enough to act on	2	In product	Deciding whether the request can go forward as it stands or whether something essential is missing. That judgement takes domain knowledge the requester does not have.
1.3	Gather the information still required from the requester	2	In product	Getting hold of what is missing. Today this happens by email and the answers sit in an inbox; the point of doing it here is to make the exchange part of the request.
1.3.1	Ask the requester a question against the request	3	In product	Raising a specific query where the requester can see it, attached to the request and available to whoever handles the request next.
1.3.2	Identify the medium required to input the missing data	3	In product	Working out what form the missing information has to take — a marked plan, a photograph, a quantity, an approval — before asking for it.
1.3.3	Input data and files and confirm the requirement with the requester	3	In product	Recording what was supplied and confirming with the requester that it describes the need correctly, so that the request is agreed before work starts.
1.4	Identify duplicate requests for the same need	2	In product	Recognising that two requests describe the same furniture need, which happens when more than one authorized requester acts on the same report.
1.4.1	Determine which request continues	3	In product	Choosing which request carries the work forward, normally the earlier or the more complete of the two.
1.4.2	Preserve the history of the request that continues	3	In product	Making sure that resolving a duplicate does not throw away information, decisions or attachments recorded on either request.
2	Decide how the need will be met	1	Partly outside	Establishing the supply path. This decides cost, timescale and which groups get involved, and it is where a request commits to the recycled route or the new one.
2.1	Check recycle-program inventory for a suitable item	2	Partly outside	Establishing whether suitable furniture already exists in government stock, honouring the requester's stated preference where they gave one.
2.2	Establish whether the item must be purchased new	2	Partly outside	Concluding that no suitable recycled item exists and that something has to be bought, which moves the request onto the procurement path.
2.3	Record the approval where one is required	2	Partly outside	Capturing the approval and the decision information that comes with it, so that the authority for the spend is part of the record rather than a separate email.
2.4	Link the purchase order reference to the request	2	Partly outside	Attaching the procurement reference for a furniture purchase to the request, so that the request and the order it produced can be read together without going into the procurement system separately.
3	Coordinate the work	1	Partly outside	Moving the request through the several groups and suppliers that carry out the physical work. This is where the problem statement locates the difficulty: information gets lost at the handoffs.

No.	Task	Level	Where performed	Description
3.1	Assign the request to the responsible group or coordinator	2	In product	Establishing explicit ownership, so that the requester and other staff can tell who is holding the request.
3.2	Hand the request on to the next group without losing its history	2	In product	Passing ownership on while keeping the accumulated information, decisions and conversation attached to the request.
3.3	Record progress against the fulfilment stages	2	In product	Updating the stage the request has reached, which is what makes status mean something to the requester rather than being nominal.
3.4	Record external supplier, delivery and installation progress	2	Partly outside	Reflecting what suppliers and installers are doing. Staff enter it rather than the suppliers themselves, because outside parties do not transact in the system.
3.5	Schedule the delivery, move or installation	2	Partly outside	Setting the date the physical work will happen and making it visible to the requester, who often has to arrange access or clear the area.
4	Keep the client informed	1	In product	The service side of the following-up the requester is doing. Each of these tasks exists to remove a chase the requester would otherwise have to make.
4.1	Update the status the requester sees	2	In product	Making sure the state visible to the client reflects reality, which is the single change most likely to reduce enquiries.
4.2	Respond to the requester against the request	2	In product	Answering questions on the request rather than in a separate message, so that the answer is there for everyone who needs it later.
4.3	Notify the requester when action is needed from them	2	In product	Alerting the requester specifically when the request is waiting on them, which is the only notification a busy requester reliably wants.
5	Close and account for the work	1	In product	Finishing the request properly: confirming the outcome, recording why it closed, and making the request available to the reporting the service is measured by.
5.1	Confirm the work was completed	2	Partly outside	Establishing that the furniture is in place and that the request has been fulfilled as agreed.
5.2	Record the asset and warranty information for what was supplied	2	In product	Capturing the manufacturer, model or serial number of the furniture actually supplied, together with any warranty period or expiry date. This is what makes a warranty claim possible years later, and it is done once, by the person who has the information in front of them.
5.3	Record the closure and any remaining issue	2	In product	Classifying how the request ended and capturing anything the requester reported as outstanding, so that closure is not a way of hiding an unresolved problem.
5.4	Report on requests by location, status and period	2	In product	Producing the operational view across many requests that service managers need for capacity, cost and performance, using the information the requests already hold.